

微信小程序uni-app：onLoad获取参数并请求接口的实现与注意事项

一、需求分析

在微信小程序的 `Unit-App`（推测为 `uni-app` 笔误）中，页面通过 `onLoad` 生命周期获取跳转传递的参数，再将该参数作为接口请求参数，获取对应数据并处理。

二、核心实现步骤

1. 页面跳转传递参数（跳转页）：通过 `uni.navigateTo` / `uni.redirectTo` 等API传递参数；
2. 目标页onLoad接收参数：`onLoad` 生命周期的 `options` 参数接收传递的值；
3. 封装接口请求：基于 `uni.request` 封装请求函数，接收参数并发起请求；
4. 请求数据并渲染：在 `onLoad` 中调用接口，处理返回数据并绑定到页面数据。

三、完整代码示例

1. 跳转页（如 index.vue）

Code block

```
1  <template>
2    <view class="container">
3      <!-- 点击跳转并传递参数（示例：商品ID） -->
4      <button @click="goToDetail(1001)">查看商品详情</button>
5    </view>
6  </template>
7
8  <script>
9    export default {
10      methods: {
11        goToDetail(goodsId) {
12          uni.navigateTo({
13            // 传递参数 goodsId 到详情页
14            url: `/pages/goods/detail?goodsId=${goodsId}`
15          });
16        }
17      }
18    };
```

2. 目标页（如 pages/goods/detail.vue）

Code block

```
1  <template>
2    <view class="detail-container">
3      <!-- 加载状态 -->
4      <view v-if="loading" class="loading">加载中...</view>
5
6      <!-- 数据渲染 -->
7      <view v-else-if="goodsInfo" class="goods-info">
8        <view class="title">{{ goodsInfo.name }}</view>
9        <view class="price">¥{{ goodsInfo.price }}</view>
10       <view class="desc">{{ goodsInfo.desc }}</view>
11     </view>
12
13     <!-- 无数据/错误状态 -->
14     <view v-else class="empty">暂无商品信息</view>
15   </view>
16 </template>
17
18 <script>
19 // 引入接口封装文件（推荐），也可直接在页面内定义请求函数
20 import { getGoodsDetail } from '@api/goods.js';
21
22 export default {
23   data() {
24     return {
25       loading: false, // 加载状态
26       goodsId: '', // 接收的参数
27       goodsInfo: null // 接口返回的商品信息
28     };
29   },
30   onLoad(options) {
31     // 1. 接收跳转传递的参数（options 包含所有传递的键值对）
32     this.goodsId = options.goodsId;
33     console.log('接收的商品ID: ', this.goodsId);
34
35     // 2. 调用接口获取数据（需判断参数是否有效）
36     if (this.goodsId) {
37       this.getGoodsData();
38     } else {
39       uni.showToast({
40         title: '参数错误',
```

```
41         icon: 'none'
42     });
43 }
44 },
45 methods: {
46     // 获取商品详情数据
47     async getGoodsData() {
48         try {
49             this.loading = true;
50             // 3. 调用接口，将接收的参数作为请求参数传递
51             const res = await getGoodsDetail(this.goodsId);
52
53             // 4. 处理接口返回数据
54             if (res.code === 200) {
55                 this.goodsInfo = res.data;
56             } else {
57                 uni.showToast({
58                     title: res.msg || '获取数据失败',
59                     icon: 'none'
60                 });
61             }
62         } catch (error) {
63             console.error('请求失败: ', error);
64             uni.showToast({
65                 title: '网络异常，请重试',
66                 icon: 'none'
67             });
68         } finally {
69             this.loading = false; // 关闭加载状态
70         }
71     }
72 }
73 };
74 </script>
75
76 <style scoped>
77 .detail-container {
78     padding: 20rpx;
79 }
80 .loading {
81     text-align: center;
82     padding: 40rpx;
83     color: #999;
84 }
85 .goods-info {
86     margin-top: 20rpx;
87 }
```

3. 接口封装文件（如 api/goods.js）

Code block

```
1  // 封装请求基地址（可在全局配置中定义）
2  const baseUrl = 'https://api.example.com';
3
4  /**
5   * 获取商品详情接口
6   * @param {string/number} goodsId - 商品ID
7   * @returns {Promise} 请求Promise
8   */
9  export const getGoodsDetail = (goodsId) => {
10    return new Promise((resolve, reject) => {
11      uni.request({
12        url: `${baseUrl}/goods/detail`, // 接口地址
13        method: 'GET', // 请求方式：GET/POST 等
14        data: {
15          goodsId: goodsId // 将参数传递给接口
16        },
17        success: (res) => {
18          // 统一处理接口返回
19          if (res.statusCode === 200) {
20            resolve(res.data);
21          } else {
22            reject(new Error(`请求失败: ${res.statusCode}`));
23          }
24        },
25        fail: (err) => {
26          reject(err);
27        }
28      });
29    });
30  };
```

四、关键注意事项

1. 参数类型处理： `onLoad` 接收的参数均为字符串类型，若接口需要数字类型（如ID），需手动转换：

Code block

```
1  this.goodsId = Number(options.goodsId); // 转换为数字
```

2. **参数校验**：必须判断参数是否存在/有效，避免传递空参数导致接口请求异常；
3. **异步处理**：接口请求为异步操作，需使用 `async/await` 或 `Promise` 处理，避免数据未返回时渲染错误；
4. **异常捕获**：通过 `try/catch` 捕获网络错误、接口返回错误，提升用户体验；
5. **复杂参数传递**：若传递对象/数组等复杂数据，需先通过 `JSON.stringify` 序列化，接收时 `JSON.parse` 解析：

Code block

```
1  // 跳转页
2  const params = { id: 1001, name: '商品A' };
3  uni.navigateTo({
4    url: `/pages/detail?params=${encodeURIComponent(JSON.stringify(params))}`
5  });
6
7  // 接收页 onLoad
8  const params = JSON.parse(decodeURIComponent(options.params));
```

6. **跨平台适配**：uni-app 支持多端，`onLoad` 生命周期在H5/小程序/APP端均有效，无需额外适配。

五、扩展场景

- **参数来自其他渠道**：若参数来自 `uni.$on` 全局事件、本地存储（`uni.getStorage`）等，可在 `onLoad` 中统一获取后调用接口；
- **下拉刷新/重新加载**：可将接口请求逻辑封装为方法，在 `onPullDownRefresh` 中复用：

Code block

```
1  onPullDownRefresh() {
2    this.getGoodsData().finally(() => {
3      uni.stopPullDownRefresh(); // 停止下拉刷新
4    });
5  }
```

通过以上实现，可完成从 `onLoad` 获取参数并作为接口参数请求数据的核心需求，同时保证代码的可维护性和用户体验。

（注：文档部分内容可能由 AI 生成）